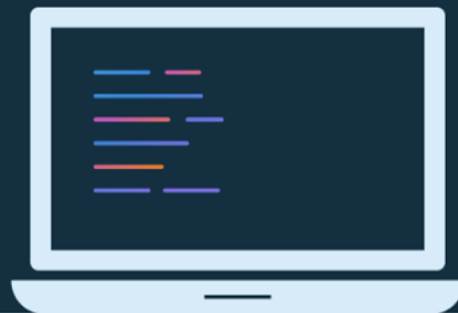




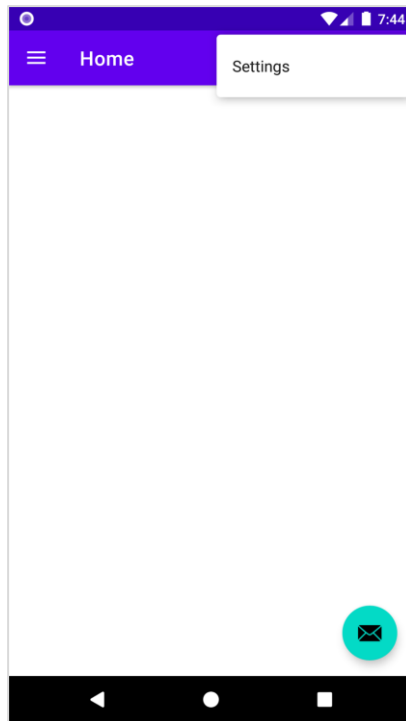
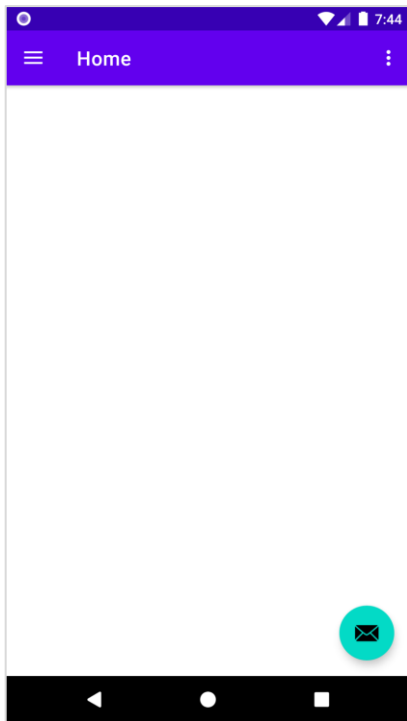
Android Development with Kotlin



App bar, navigation drawer, and menus

App Bar with Options Menu

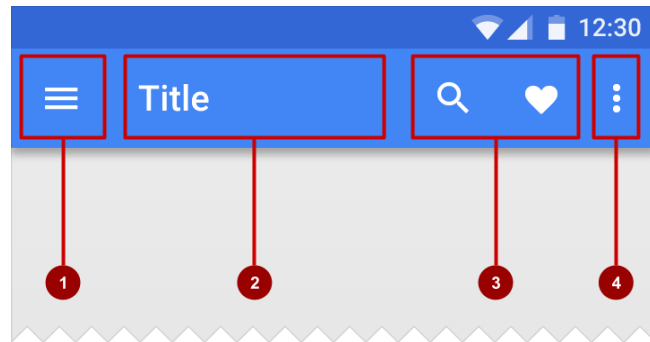
App bar



App bar

The ***app bar***, also known as the ***action bar***, is the bar at the top of each screen - same for all devices (usually)

1. Nav icon to open the navigation drawer
2. Title of Current Activity
3. Icons for options menu items
4. Action overflow button
for the rest of the options menu



Setup the App bar

1. Make sure the activity extends [AppCompatActivity](#)

Kotlin

Java

```
public class MyActivity extends AppCompatActivity {  
    // ...  
}
```



Note: Make this change for every activity in your app that uses a [Toolbar](#) as an app bar.

Setup the App bar cont'd

2. Make sure that the theme of the app uses one of the component's [NoActionBar](#) themes.

Using one of these themes prevents the app from using the native [ActionBar](#) class to provide the app bar.

```
<application
    android:theme="@style/Theme.AppCompat.Light.NoActionBar"
/>
```



Setup the App bar cont'd

3. Add a [Toolbar](#) to your layout
See the [Material Design specification](#)
for recommendations regarding app bar elevation.

Position the toolbar
at the top of the activity's [layout](#),
since you are using it as an app bar.

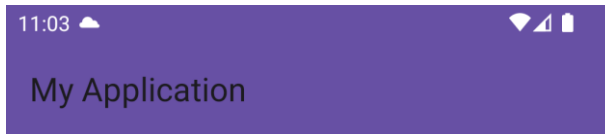
```
<androidx.appcompat.widget.Toolbar  
    android:id="@+id/toolbar"  
    android:layout_width="match_parent"  
    android:layout_height="wrap_content"  
    android:background="?attr/colorPrimary"  
    android:minHeight="?attr/actionBarSize"  
    android:theme="?attr/actionBarTheme"  
    app:layout_constraintEnd_toEndOf="parent"  
    app:layout_constraintStart_toStartOf="parent"  
    app:layout_constraintTop_toTopOf="parent" />
```


Setup the App bar cont'd

4. In the activity's `onCreate()` method, call the activity's `setSupportActionBar()` method and pass the activity's toolbar. This method sets the toolbar as the app bar for the activity

```
override fun onCreate(savedInstanceState: Bundle?) {  
    super.onCreate(savedInstanceState)  
    setContentView(R.layout.activity_main)  
  
    val toolbar : Toolbar = findViewById<Toolbar>(R.id.toolbar)  
    setSupportActionBar(toolbar)  
}
```

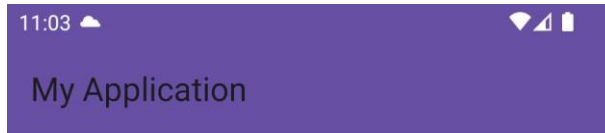
Your app now has a basic action bar.



Menu

The issue with the title color

You need to add a custom style for the Toolbar



```
<style name="AppTheme.AppBarOverlay" parent="ThemeOverlay.AppCompat.Dark.ActionBar" />
```

- The link with the Toolbar through

`android:theme="@style/AppTheme.AppBarOverlay"`



Setup the App bar cont'd

By default, the action bar contains just the name of the app and an overflow menu.

The options menu initially contains just the **Settings** item.

You can add more actions to the action bar and the overflow menu, as described in [Adding and Handling Actions](#).

To use the [ActionBar](#) utility methods, call the activity's [getSupportActionBar\(\)](#) method. For example, to change the title of the screen...

```
supportActionBar?.title = "New Title"
```

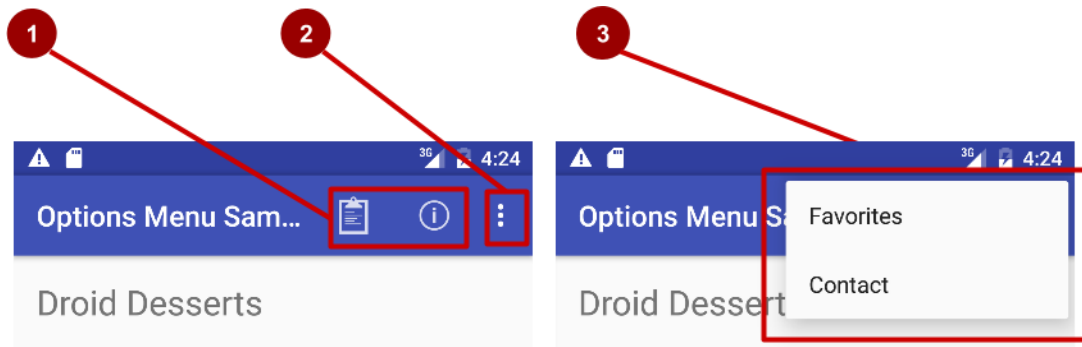


Options Menu

- The app bar allows you to add buttons for user actions.
- This feature lets you put the most important actions for the current context right at the top of the app.
- Space in the app bar is limited.
- If an app declares more actions than can fit in the app bar, the app bar send the excess actions to an overflow menu.
- The app can also specify that an action should always be shown in the overflow menu, instead of being displayed on the app bar.

Options Menu cont'd

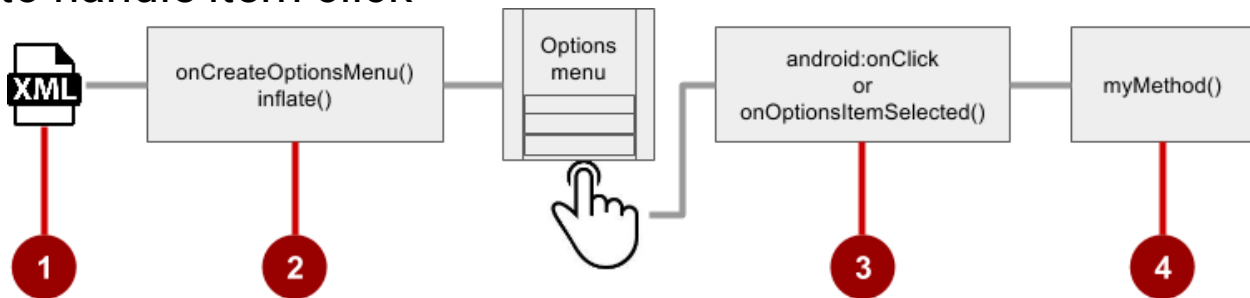
1. Action icons in the app bar for important items
2. Tap the three dots, the "action overflow button" to see the options menu
3. Appears in the right corner of the app bar
For navigating to other activities and editing app settings



Adding Options Menu

Steps to implement options menu

1. XML menu resource (`home_menu.xml`)
2. `onCreateOptionsMenu()` to inflate the menu
3. `onClick` attribute or `onOptionsItemSelected()`
4. Method to handle item click



Adding Options Menu cont'd

Create menu resource

All action buttons and other items available in the action overflow are defined in an XML [menu resource](#).

1. Create a menu resource (`res/menu`) directory
2. Create XML menu resource (`home_menu.xml`)
3. Add an entry for each menu item

Menu

Define menu items in XML menu resource (located in `res/menu` folder)

```
<menu xmlns:android="http://schemas.android.com/apk/res/android"
      xmlns:app="http://schemas.android.com/apk/res-auto">
  <item
    android:id="@+id/action_settings"
    android:orderInCategory="100"
    android:title="@string/action_settings"
    app:showAsAction="never" />
</menu>
```

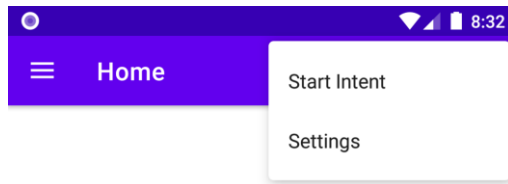

Options menu example

```
<menu xmlns:android="http://schemas.android.com/apk/res/android"
      xmlns:app="http://schemas.android.com/apk/res-auto">

    <item android:id="@+id/action_intent"
          android:title="@string/action_intent" />

    <item
        android:id="@+id/action_settings"
        android:orderInCategory="100"
        android:title="@string/action_settings"
        app:showAsAction="never" />

</menu>
```



Inflate options menu

```
override fun onCreateOptionsMenu(menu: Menu): Boolean {  
    menuInflater.inflate(R.menu.main, menu)  
    return true  
}
```

Handle menu options selected

```
override fun onOptionsItemSelected(item: MenuItem): Boolean {  
    when (item.itemId) {  
        R.id.action_intent -> {  
            val intent = Intent(Intent.ACTION_WEB_SEARCH)  
            intent.putExtra(SearchManager.QUERY, "pizza")  
            if (intent.resolveActivity(packageManager) != null) {  
                startActivity(intent)  
            }  
        }  
        else -> Toast.makeText(this, item.title, Toast.LENGTH_LONG).show()  
        ...  
    }  
}
```

Menu

12:18

New Title

Start Intent

Settings

The issue with the option menu items (text white on white background)

- You need to add a custom style for the options menu text

```
<style name="OptionsMenuText" parent="android:TextAppearance.Widget.PopupMenu.Small">
    <item name="android:textColor">@android:color/black</item>
</style>
```

- Then link with the Toolbar through popupTheme

12:26

New Title

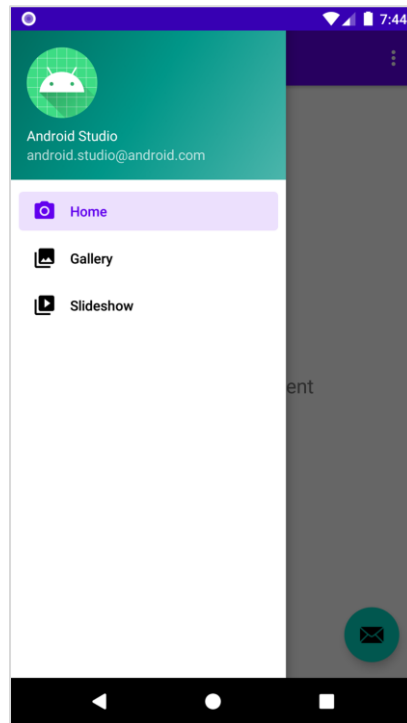
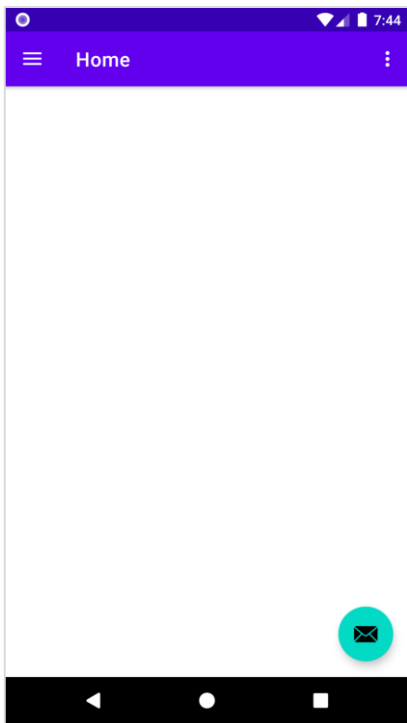
Start Intent

Settings

```
<androidx.appcompat.widget.Toolbar
    android:id="@+id/toolbar"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:background="?attr/colorPrimary"
    android:minHeight="?attr/actionBarSize"
    android:theme="@style/AppTheme.AppBarOverlay"
    app:popupTheme="@style/OptionsMenuText"
    app:layout_constraintEnd_toEndOf="parent"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintTop_toTopOf="parent" />
```

Navigation Drawer

Navigation drawer



Navigation drawer cont'd

A navigation drawer

is a UI panel that shows your app's main navigation menu.

The drawer appears

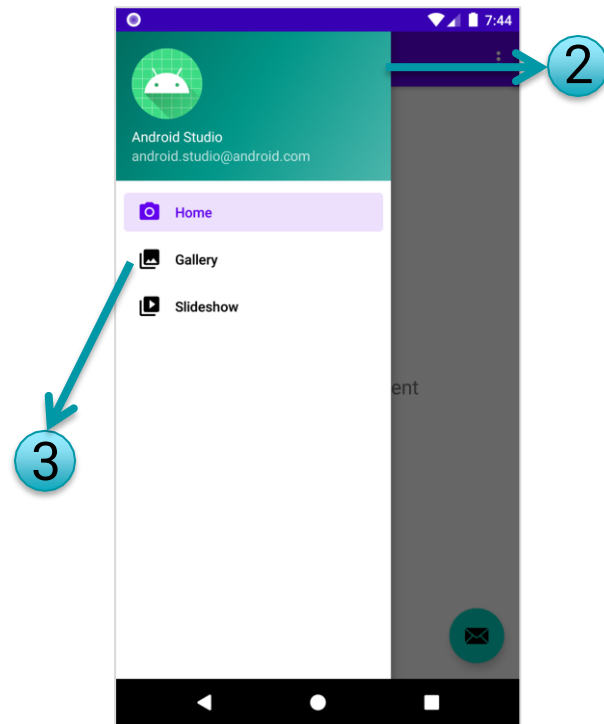
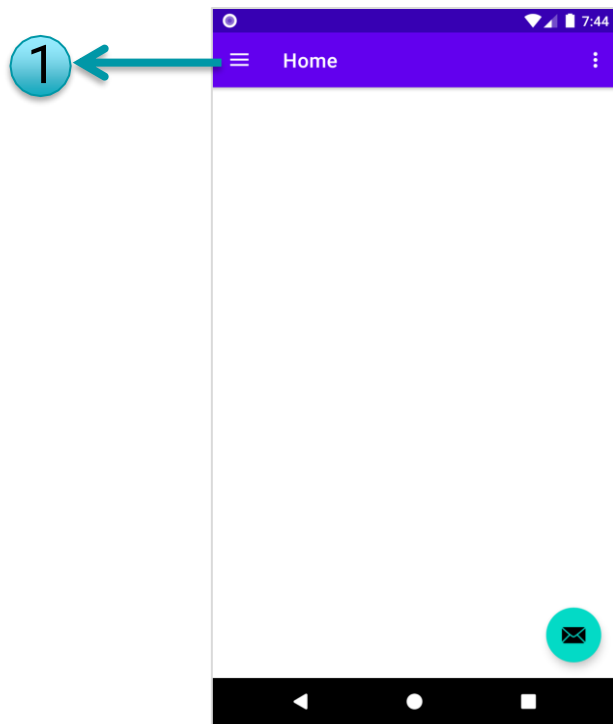
when the user touches the drawer icon in the app bar

or when the user swipes a finger from the left edge of the screen.

Navigation drawer cont'd

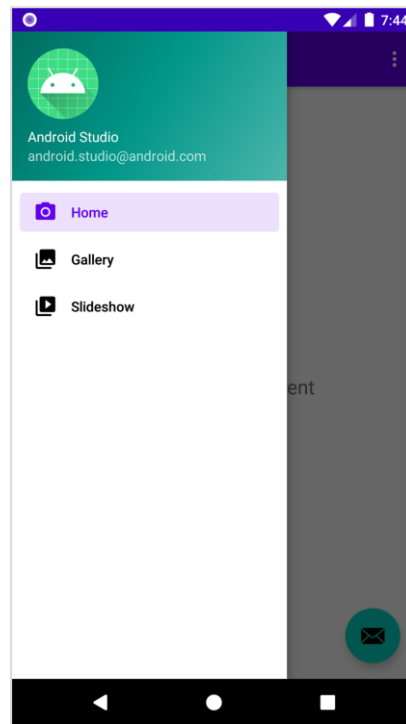
Descendant navigation

1. Icon in the app bar
2. Header
3. Menu items



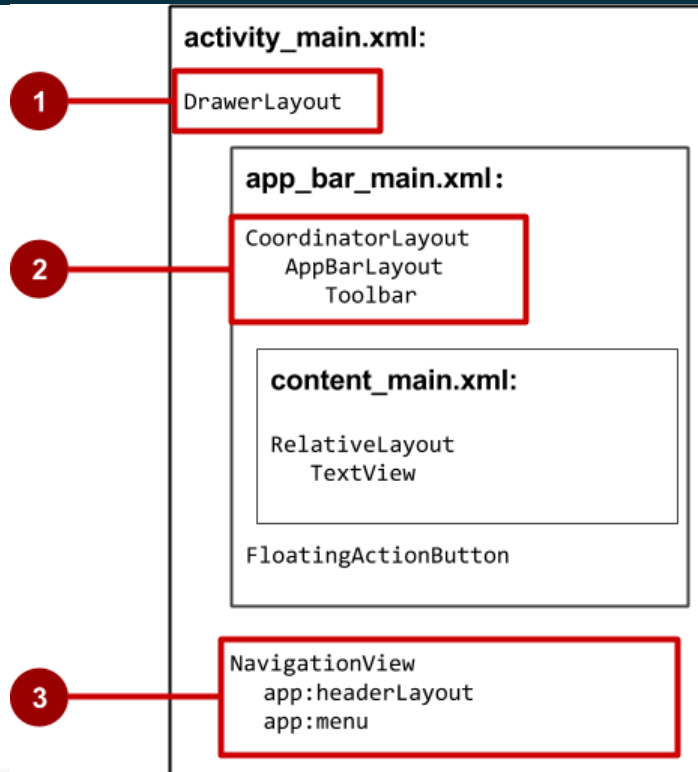
Make a Navigation Drawer

- A **navigation drawer** as the *Activity* layout root ***ViewGroup***
- A ***NavigationView*** for the drawer itself
- An app bar layout that includes room for a navigation icon button
- A ***content layout*** for the ***Activity*** that displays the navigation drawer
- A layout for the navigation drawer header



Navigation Drawer Activity Layout

1. [DrawerLayout](#) is the root **ViewGroup** of the **Activity** layout
2. CoordinatorLayout contains app bar layout with a Toolbar
3. [NavigationView](#) with layouts for header and selectable items



Steps to Implement Navigation Drawer

1. Populate the navigation drawer menu with item titles and icons
2. Set up navigation drawer and item listeners in the **Activity** code
3. Handle the navigation menu item selections

Creating the Navigation Drawer Layout

1. A *DrawerLayout* as the root of the *Activity* layout in *activity_main.xml*
2. The main content of the screen is defined in the *app_bar_main.xml* layout file
3. [NavigationView](#) that represents a standard navigation menu that can be populated by a menu resource XML file.

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.drawerlayout.widget.DrawerLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:id="@+id/drawer_layout"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:fitsSystemWindows="true"
    tools:context=".MainActivity"
    tools:openDrawer="start">
```

```
<include layout="@layout/app_bar_main"
    android:layout_width="match_parent"
    android:layout_height="match_parent" />
```

```
<com.google.android.material.navigation.NavigationView
    android:id="@+id/nav_view"
    android:layout_width="wrap_content"
    android:layout_height="match_parent"
    android:layout_gravity="start"
    android:fitsSystemWindows="true"
    app:menu="@menu/main_drawer" />
```

```
</androidx.drawerlayout.widget.DrawerLayout>
```

activity_main.xml

App bar layout

1. The *app_bar_main.xml* layout uses a *CoordinatorLayout* as its root and includes the *content_main.xml* layout.
2. The *app_bar_main.xml* layout uses the *android:fitsSystemWindows="true"* attribute to set the padding of the app bar to ensure that it doesn't overlay the system windows such as the status bar.

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.coordinatorlayout.widget.CoordinatorLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:fitsSystemWindows="true">
```

```
<com.google.android.material.appbar.AppBarLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:theme="@style/AppTheme.AppBarOverlay">
```

```
<androidx.appcompat.widget.Toolbar
    android:id="@+id/toolbar"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:background="?attr/colorPrimary"
    android:minHeight="?attr/actionBarSize"
    android:theme="@style/AppTheme.AppBarOverlay"
    app:layout_constraintEnd_toEndOf="parent"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintTop_toTopOf="parent"
    app:popupTheme="@style/OptionsMenuText" />
```

```
</com.google.android.material.appbar.AppBarLayout>
```

```
<include layout="@layout/content_main" />
```

```
</androidx.coordinatorlayout.widget.CoordinatorLayout>
```

app_bar_main.xml

The content layout for the main activity screen

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    xmlns:app="http://schemas.android.com/apk/res-auto">

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Main Content fot Home"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent" />
</androidx.constraintlayout.widget.ConstraintLayout>
```

content_main.xml



More menu options

```
<menu>
  <group android:checkableBehavior="single">
    <item
      android:id="@+id/nav_home"
      android:icon="@drawable/ic_menu_camera"
      android:title="@string/menu_home" />
    <item
      android:id="@+id/nav_gallery"
      android:icon="@drawable/ic_menu_gallery"
      android:title="@string/menu_gallery" />
    <item
      android:id="@+id/nav_slideshow"
      android:icon="@drawable/ic_menu_slideshow"
      android:title="@string/menu_slideshow" />
  </group>
</menu>
```

Setting up the navigation drawer and item listener

To use a listener for the navigation drawer's menu items, the *activity* hosting the navigation drawer must implement the *OnNavigationItemSelectedListener* interface:

1. Implement *NavigationView.OnNavigationItemSelectedListener* in the class definition:

```
class MainActivity : AppCompatActivity(), NavigationView.OnNavigationItemSelectedListener {
```


Setting up the navigation drawer and item listener

To use a listener for the navigation drawer's menu items, the *activity* hosting the navigation drawer must implement the *OnNavigationItemSelectedListener* interface:

2. Implement the needed methods:

```
override fun onNavigationItemSelectedListener(item: MenuItem): Boolean {  
    TODO( reason: "Not yet implemented")  
}
```

Setting up the navigation drawer and item listener

To use a listener for the navigation drawer's menu items, the *activity* hosting the navigation drawer must implement the *OnNavigationItemSelectedListener* interface:

3. Instantiate the *DrawerLayout* and *NavigationView* objects:

```
private fun instantiateDrawerLayout() {  
    val drawerLayout: DrawerLayout = findViewById(R.id.drawer_layout)  
    val toggle = ActionBarDrawerToggle( activity: this, drawerLayout, toolbar, R.string.navigation_drawer_open, R.string.navigation_drawer_close)  
  
    drawerLayout.addDrawerListener(toggle)  
    toggle.syncState()  
  
    val navigationView: NavigationView = findViewById(R.id.nav_view)  
    navigationView.setNavigationItemSelectedListener(this)  
}
```

Setting up the navigation drawer and item listener

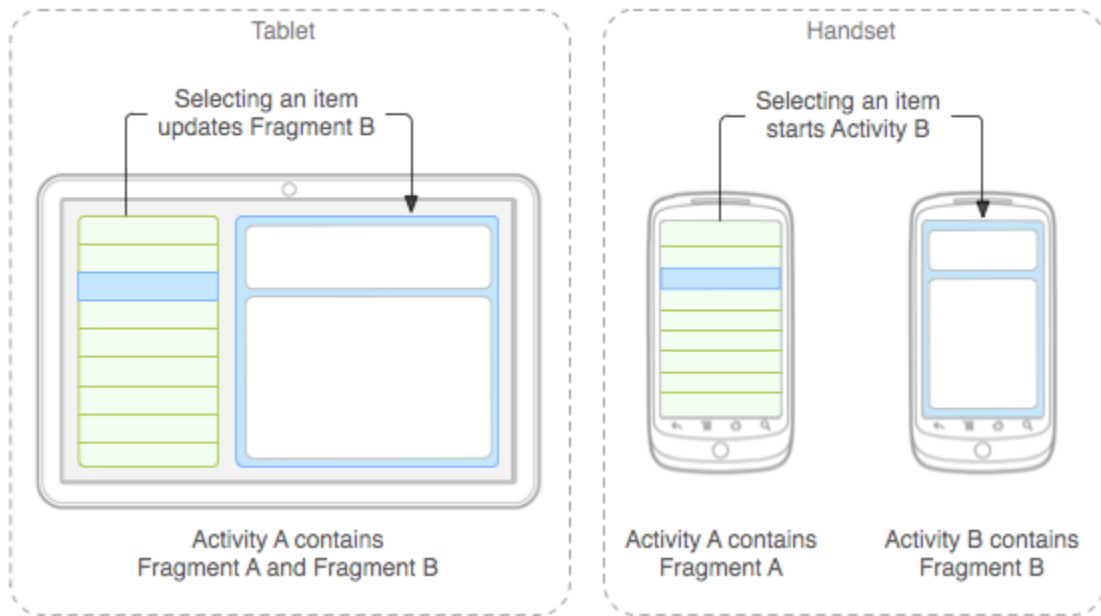
To use a listener for the navigation drawer's menu items, the *activity* hosting the navigation drawer must implement the *OnNavigationItemSelectedListener* interface:

4. Handle navigation menu item selection:

```
override fun onNavigationItemSelectedListener(item: MenuItem): Boolean {  
    when (item.itemId) {  
        R.id.nav_home -> {  
            drawerLayout.closeDrawer(GravityCompat.START)  
            // do action  
            return true  
        }  
        R.id.nav_gallery -> {  
            drawerLayout.closeDrawer(GravityCompat.START)  
            // do action  
            return true  
        }  
        R.id.nav_slideshow -> {  
            drawerLayout.closeDrawer(GravityCompat.START)  
            // do action  
            return true  
        }  
        else -> return false  
    }  
}
```

Fragments

Fragments for tablet layouts



Fragment

- Represents a behavior or portion of the UI in an activity ("microactivity")
- Must be hosted in an activity
- Lifecycle tied to host activity's lifecycle
- Can be added or removed at runtime

Note about fragments

Use the AndroidX version of the `Fragment` class.
(`androidx.fragment.app.Fragment`).

Don't use the platform version of the `Fragment` class
(`android.app.Fragment`), which was deprecated.

Gradle: Building an Android app

What is Gradle?

- Builds automation system
- Manages the build cycle via a series of tasks
(for example, compiles Kotlin sources, runs tests, installs app to device)
- Determines the proper order of tasks to run
- Manages dependencies
between projects and third-party libraries

Gradle build file

- Declare plugins
- Define Android properties
- Handle dependencies
- Connect to repositories

Plugins (Old)

Provide libraries and infrastructure needed by your app

```
apply plugin: 'com.android.application'
```

```
apply plugin: 'kotlin-android'
```

```
apply plugin: 'kotlin-android-extensions'
```

Plugins (New)

Provide libraries and infrastructure needed by your app

```
Plugins {  
    id 'com.android.application' version '8.0.1' apply false  
    id 'com.android.library' version '8.0.1' apply false  
    id 'org.jetbrains.kotlin.android' version '1.8.20' apply false  
}
```

Plugins (Newest)

libs.versions.toml (Version Catalog)

```
[versions]
```

```
agp = "8.5.1"
```

```
kotlin = "1.9.21"
```

```
[plugins]
```

```
android-application =
```

```
{ id = "com.android.application", version.ref = "agp" }
```

```
jetbrains-kotlin-android =
```

```
{ id = "org.jetbrains.kotlin.android", version.ref = "kotlin" }
```

Plugins (Newest) cont'd

build.gradle.kts (Project: *project_name*)

```
plugins {  
    alias(libs.plugins.android.application) apply false  
    alias(libs.plugins.jetbrains.kotlin.android) apply false  
}
```

Plugins (Newest) cont'd

build.gradle.kts (Module: app)

```
plugins {  
    alias(libs.plugins.android.application)  
    alias(libs.plugins.jetbrains.kotlin.android)  
}
```

Android configuration

```
android {  
    compileSdk = 34  
  
    defaultConfig {  
        applicationId "com.example.sample"  
        minSdk = 26  
        targetSdk = 34  
    }  
}
```


Dependencies

```
dependencies {  
    implementation "org.jetbrains.kotlin:kotlin-stdlib-jdk7:$kotlin_version"  
    implementation 'androidx.core:core-ktx:1.3.2'  
    implementation 'androidx.appcompat:appcompat:1.2.0'  
    implementation 'com.google.android.material:material:1.2.1'  
    ...  
}
```

Dependencies (New)

[versions]

```
coreKtx = "1.13.1"  
appcompat = "1.6.1"  
material = "1.10.0"
```

libs.versions.toml (Version Catalog)

[libraries]

```
androidx-core-ktx = { group = "androidx.core",  
                      name = "core-ktx", version.ref = "coreKtx" }  
androidx-appcompat = { group = "androidx.appcompat",  
                       name = "appcompat", version.ref = "appcompat" }  
material = { group = "com.google.android.material",  
            name = "material", version.ref = "material" }
```

Dependencies (New)

build.gradle.ktx (Module :app)

```
dependencies {  
    implementation(libs.androidx.core.ktx)  
    implementation(libs.androidx.appcompat)  
    implementation(libs.material)  
    ...  
}
```

Repositories

```
repositories {  
    google()  
    mavenCentral()  
    gradlePluginPortal()  
}
```

Common Gradle tasks

- Clean
- Tasks
- InstallDebug

Navigation within an app

Navigation component

- Collection of libraries and tooling, including an integrated editor, for creating navigation paths through an app
- Assumes one `Activity` per graph with many `Fragment` destinations
- Consists of three major parts:
 - Navigation graph
 - Navigation Host (`NavHost`)
 - Navigation Controller (`NavController`)

Add dependencies

In `build.gradle`, under dependencies:

```
implementation "androidx.navigation:navigation-fragment-ktx:$nav_version"
```

```
implementation "androidx.navigation:navigation-ui-ktx:$nav_version"
```

[Get started with Navigation component | Android Developers](#)

Add dependencies (New)

libs.versions.toml (Version Catalog)

[versions]

navigationFragmentKtx = "2.9.7"

navigationUiKtx = "2.9.7"

[libraries]

androidx-navigation-fragment-ktx = { group = "androidx.navigation",
name = "navigation-fragment-ktx",
version.ref = "navigationFragmentKtx" }

androidx-navigation-ui-ktx = { group = "androidx.navigation",
name = "navigation-ui-ktx", version.ref = "navigationUiKtx" }

Add dependencies (New) cont'd

build.gradle.ktx (Module :app)

```
dependencies {  
    ...  
    implementation(libs.androidx.navigation.fragment.ktx)  
    implementation(libs.androidx.navigation.ui.ktx)  
    ...  
}
```

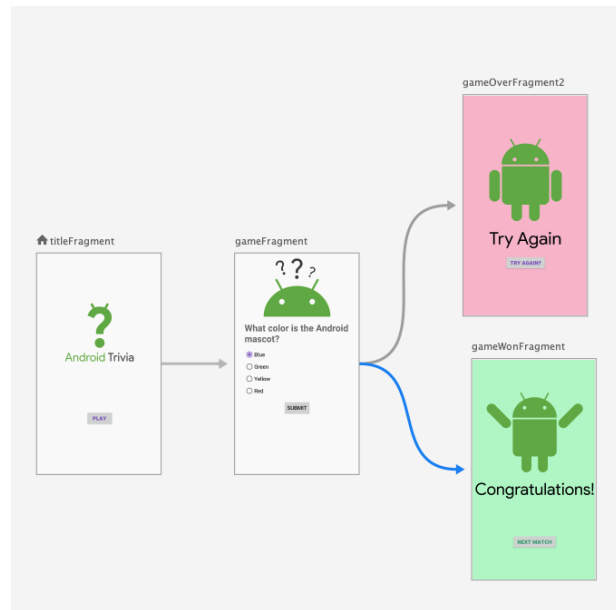
Navigation host (NavHost)

```
<androidx.fragment.app.FragmentContainerView  
    android:id="@+id/nav_host"  
    android:name="androidx.navigation.fragment.NavHostFragment"  
    android:layout_width="match_parent"  
    android:layout_height="match_parent"  
    app:defaultNavHost="true"  
    app:navGraph="@navigation/nav_graph_name"/>
```

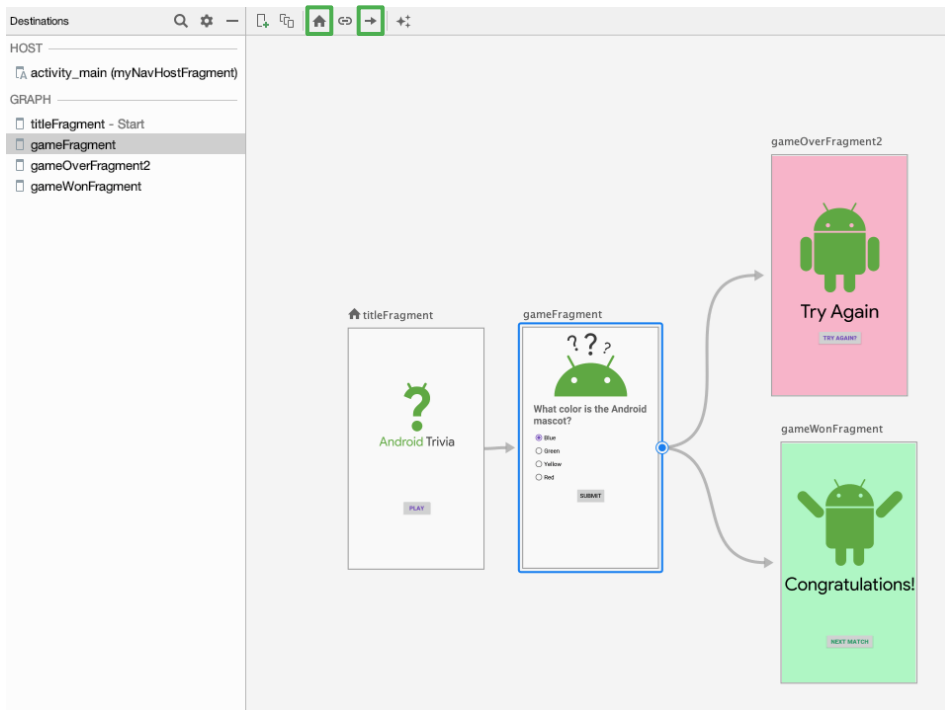
Navigation graph

New resource type located in `res/navigation` directory

- XML file containing all of your navigation destinations and actions
- Lists all the (Fragment/Activity) destinations that can be navigated to
- Lists the associated actions to traverse between them
- Optionally lists animations for entering or exiting



Navigation Editor in Android Studio



Creating a Fragment

- Extend `Fragment` class
- Override `onCreateView()`
- Inflate a layout for the Fragment that you have defined in XML

```
class DetailFragment : Fragment() {  
  
    override fun onCreateView(inflater: LayoutInflater, container: ViewGroup?,  
        savedInstanceState: Bundle?): View? {  
        return inflater.inflate(R.layout.detail_fragment, container, false)  
    }  
}
```

Specifying Fragment destinations

- Fragment destinations are denoted by the `action` tag in the navigation graph.
- Actions can be defined in XML directly or in the Navigation Editor by dragging from source to destination.
- Autogenerated action IDs take the form of `action_<sourceFragment>_to_<destinationFragment>`.

Example fragment destination

```
<fragment
    android:id="@+id/welcomeFragment"
    android:name="com.example.android.navigation.WelcomeFragment"
    android:label="fragment_welcome"
    tools:layout="@layout/fragment_welcome" >

    <action
        android:id="@+id/action_welcomeFragment_to_detailFragment"
        app:destination="@id/detailFragment" />

</fragment>
```


Navigation Controller (NavController)

`NavController` manages UI navigation in a navigation host.

- Specifying a destination path only names the action, but it doesn't execute it.
- To follow a path, use `NavController`.

Example NavController

```
class MainActivity : AppCompatActivity() {  
  
    override fun onCreate(savedInstanceState: Bundle?) {  
        ...  
        val navHostFragment =  
            supportFragmentManager.findFragmentById(R.id.fragmentContainerView)  
                                   as NavHostFragment  
        val navController = navHostFragment.navController  
        NavigationUI.setupActionBarWithNavController(this, navController)  
    }  
  
    fun navigateToDetail() {  
        navController.navigate(R.id.action_welcomeFragment_to_detailFragment)  
    }  
}
```

Example NavController

```
class WelcomeFragment : Fragment() {  
  
    override fun onViewCreated(view: View, savedInstanceState: Bundle?) {  
        ...  
        view.findViewById<Button>(R.id.btn_navigate).setOnClickListener {  
            navigateToDetail()  
        }  
    }  
  
    fun navigateToDetail() {  
        findNavController().navigate(R.id.action_welcomeFragment_to_detailFragment)  
    }  
}
```

More custom navigation behavior

Passing data between destinations

Using Safe Args:

- Ensures arguments have a valid type
- Lets you provide default values
- Generates a `<SourceDestination>Directions` class with methods for every action in that destination
- Generates a class to set arguments for every named action
- Generates a `<TargetDestination>Args` class providing access to the destination's arguments

Setting up Safe Args

In the project `build.gradle` file:

```
buildscript {  
    repositories {  
        google()  
    }  
    dependencies {  
        classpath "androidx.navigation:navigation-safe-args-gradle-plugin:$nav_version"  
    }  
}
```

In the app's or module's `build.gradle` file:

```
apply plugin: "androidx.navigation.safeargs.kotlin"
```

Setting up Safe Args (new versions)

In the project `build.gradle` file:

```
plugins {  
    id "androidx.navigation.safeargs.kotlin" version "2.9.7" apply false  
}
```

In the app's or module's `build.gradle` file:

```
plugins {  
    ...  
    id "androidx.navigation.safeargs.kotlin"  
}
```

Setting up Safe Args (newest versions)

[versions]

safeArgs = "2.9.7"

libs.versions.toml (Version Catalog)

[plugins]

safe-args-plugin = { id = "androidx.navigation.safeargs.kotlin",
version.ref = "safeArgs" }

plugins {

build.gradle.kts (Module: app)

alias(*libs.plugins.safe.args.plugin*)

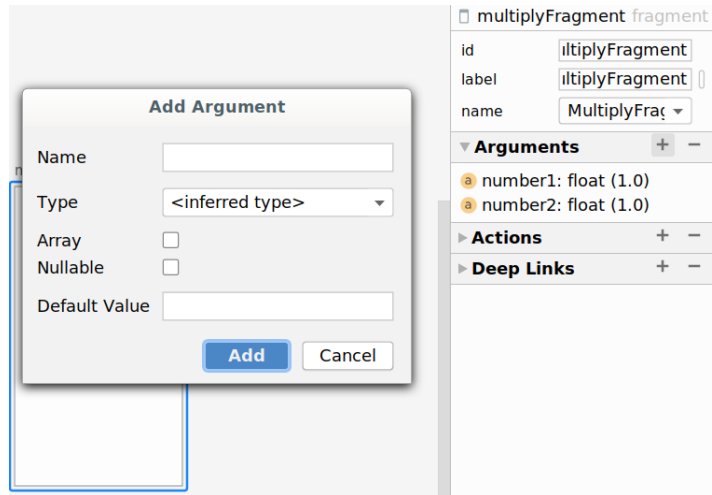
}

Sending data to a Fragment

1. Create arguments the destination fragment will expect.
2. Create action to link from source to destination.
3. Set the arguments in the action method on
`<Source>FragmentManager`.
4. Navigate according to that action using the Navigation Controller.
5. Retrieve the arguments in the destination fragment.

Destination arguments

```
<fragment
    android:id="@+id/multiplyFragment"
    android:name="com.example.arithmetic.MultiplyFragment"
    android:label="MultiplyFragment" >
    <argument
        android:name="number1"
        app:argType="float"
        android:defaultValue="1.0" />
    <argument
        android:name="number2"
        app:argType="float"
        android:defaultValue="1.0" />
</fragment>
```



Supported argument types

Type	Type Syntax <code>app:argType=<type></code>	Supports Default Values	Supports Null Values
Integer	<code>"integer"</code>	Yes	No
Float	<code>"float"</code>	Yes	No
Long	<code>"long"</code>	Yes	No
Boolean	<code>"boolean"</code>	Yes (<code>"true"</code> or <code>"false"</code>)	No
String	<code>"string"</code>	Yes	Yes
Array	above type + <code>"[]"</code> (for example, <code>"string[]"</code> <code>"long[]"</code>)	Yes (only <code>"@null"</code>)	Yes
Enum	Fully qualified name of the enum	Yes	No
Resource reference	<code>"reference"</code>	Yes	No

Supported argument types: Custom classes

Type	Type Syntax <code>app:argType=<type></code>	Supports Default Values	Supports Null Values
Serializable	Fully qualified class name	Yes (only " <code>@null</code> ")	Yes
Parcelable	Fully qualified class name	Yes (only " <code>@null</code> ")	Yes

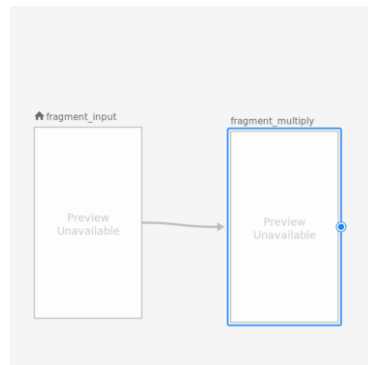
Create action from source to destination

In `nav_graph.xml`:

```
<fragment
    android:id="@+id/fragment_input"
    android:name="com.example.arithmetic.InputFragment">

    <action
        android:id="@+id/action_to_multiplyFragment"
        app:destination="@id/multiplyFragment" />

</fragment>
```



Navigating with actions

In `InputFragment.kt`:

```
override fun onCreateView(view: View, savedInstanceState: Bundle?) {  
    super.onCreateView(view, savedInstanceState)  
    binding.button.setOnClickListener {  
        val n1 = binding.number1.text.toString().toFloatOrNull() ?: 0.0  
        val n2 = binding.number2.text.toString().toFloatOrNull() ?: 0.0  
  
        val action = InputFragmentDirections.actionToMultiplyFragment(n1, n2)  
        view.findNavController().navigate(action)  
    }  
}
```

Retrieving Fragment arguments

```
class MultiplyFragment : Fragment() {  
    val args: MultiplyFragmentArgs by navArgs()  
    lateinit var binding: FragmentMultiplyBinding  
    override fun onCreateView(view: View, savedInstanceState: Bundle?) {  
        super.onCreateView(view, savedInstanceState)  
        val number1 = args.number1  
        val number2 = args.number2  
        val result = number1 * number2  
        binding.output.text = "${number1} * ${number2} = ${result}"  
    }  
}
```

Navigation UI

Menus revisited

```
override fun onOptionsItemSelected(item: MenuItem): Boolean {  
    val navController = findNavController(R.id.nav_host_fragment)  
    return item.onNavDestinationSelected(navController) ||  
        super.onOptionsItemSelected(item)  
}
```

DrawerLayout for navigation drawer

```
<androidx.drawerlayout.widget.DrawerLayout
    android:id="@+id/drawer_layout" ...>

    <fragment
        android:name="androidx.navigation.fragment.NavHostFragment"
        android:id="@+id/nav_host_fragment" ... />

    <com.google.android.material.navigation.NavigationView
        android:id="@+id/nav_view"
        app:menu="@menu/activity_main_drawer" ... />

</androidx.drawerlayout.widget.DrawerLayout>
```

Finish setting up navigation drawer

Connect `DrawerLayout` to navigation graph:

```
val appBarConfiguration = AppBarConfig(navController.graph, drawer)
```

Set up `NavigationView` for use with a `NavController`:

```
val navView = findViewById<NavigationView>(R.id.nav_view)  
navView.setupWithNavController(navController)
```

Understanding the back stack

State 1



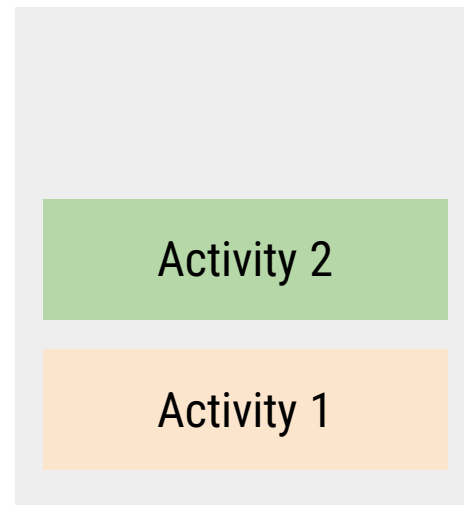
Back stack

State 2



Back stack

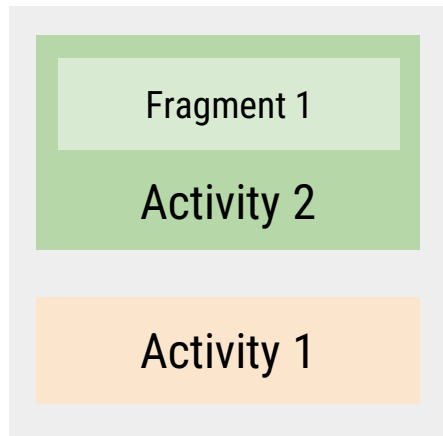
State 3



Back stack

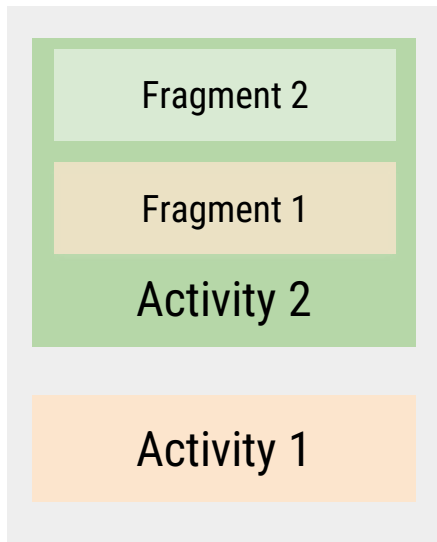
Fragments and the back stack

State 1



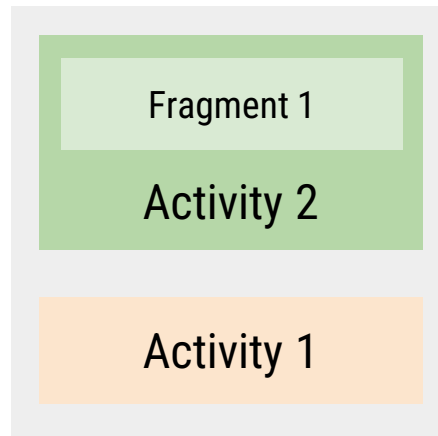
Back stack

State 2



Back stack

State 3



Back stack

Demo

Assignment 1

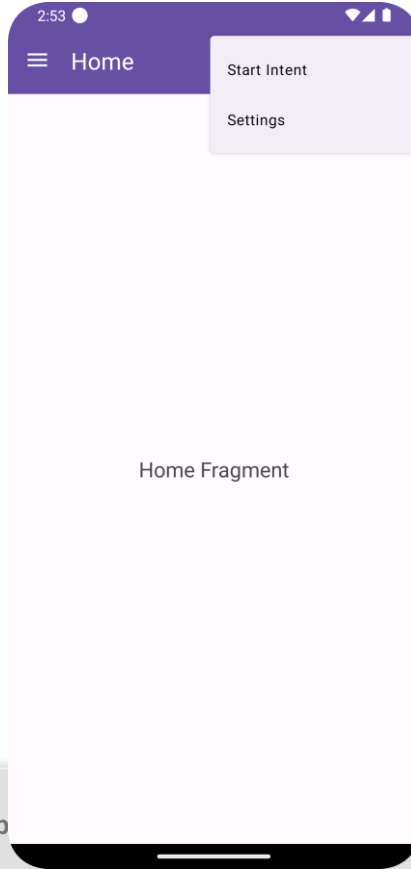
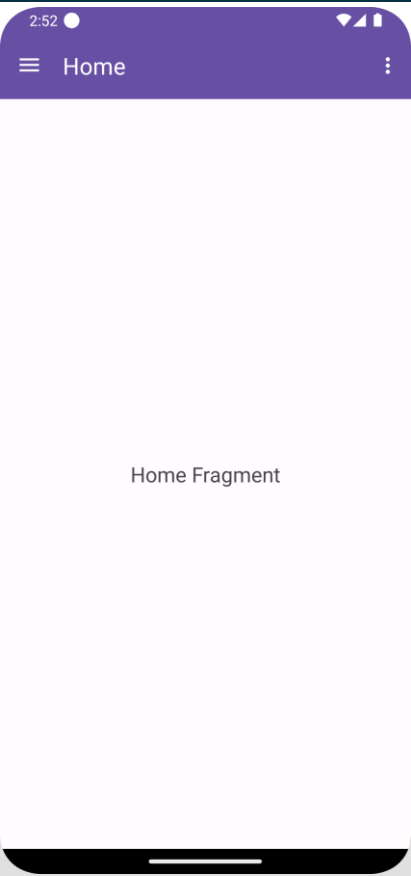
Create an app that displays menu options containing 2 options

- Start Intent: navigate to another activity
- Settings: Open Settings Fragment

and Navigation Drawer with 3 menu items

- Home: Open Home Fragment
- Gallery: Open Gallery Fragment
- Slideshow: Open Slideshow Fragment

Assignment 1



Assignment 2

Nav graph